

Documentação

1. Sugestão de Quantidade e Escopo

Para um sistema de PDV e Gestão como o SaborRush, recomendo a criação de 3 Diagramas de Atividade Principais.

Essa quantidade é ideal porque cobre os três pilares críticos do negócio sem se tornar repetitiva:

Fluxo de Venda (Front-end): Do pedido à cozinha (foco no Garçom).

Fluxo Financeiro (Checkout): Do pagamento ao fechamento de mesa (foco no Caixa).

Fluxo de Produção (Back-end Operacional): Da preparação à entrega (foco na Cozinha).

Essa divisão mapeia perfeitamente os módulos citados no slide 3 do documento ("Gestão, Vendas, Estoque, Produção").

2. Geração dos Diagramas

Abaixo estão os diagramas sugeridos, utilizando a sintaxe Mermaid (que pode ser renderizada visualmente). Eles foram desenhados considerando as "raias" (swimlanes) para separar as responsabilidades, o que se alinha bem com a sua intenção de usar Clean Architecture (separando UI, Regras de Negócio e Banco de Dados/Infra).

Diagrama 1: Processo de Realização de Pedido

Foco: Módulo I (Pedidos e Atendimento). Este diagrama cobre a abertura da mesa [RF14], a escolha dos itens e a verificação automática de estoque [RF02].

Snippet de código

```
graph TD
  %% Atores e Raias
  subgraph Garcom_UI [Garçom / App Next.js]
    A([Início]) --> B[Selecionar Mesa/Comanda]
    B --> C{Mesa Aberta?}
    C -- Não --> D[Abrir Nova Mesa/Comanda]
    C -- Sim --> E[Adicionar Itens ao Pedido]
    D --> E
    E --> F[Confirmar Pedido]
    F --> J[Receber Confirmação Visual]
    J --> K([Fim])
  end

  subgraph Sistema_Backend [Sistema / API]
    F --> G{Estoque Disponível?}
    G -- Não --> H[Retornar Erro: Item Indisponível]
    H --> E
  end
```

```

    G -- Sim --> I[Registrar Pedido e Baixar Estoque]
    I --> L[Disparar Ordem para Cozinha]
    L --> J
end

```

```

subgraph Cozinha_Display [Monitor da Cozinha]
    L --> M[Exibir 'Novo Pedido']
end

```

Pontos Chave Deste Diagrama:

Validação de Regra de Negócio: A verificação de estoque (Estoque Disponível?) ocorre no Backend, garantindo a integridade dos dados.

Integração: Mostra a comunicação entre o app do garçom e o display da cozinha.

Diagrama 2: Pagamento e Fechamento de Conta

Foco: Módulo II (Pagamentos e Financeiro). Cobre a aplicação de taxas [RF10], cupons [RF15] e o fechamento [RF07].

Snippet de código

```

graph TD
    %% Atores e Raias
    subgraph Cliente_Atore [Cliente]
        Start([Solicitar Conta]) --> Await([Aguardar Conferência])
        Pay[Realizar Pagamento] --> Leave([Fim])
    end
end

```

```

subgraph Caixa_UI [Operador de Caixa / PDV]
    Await --> B[Visualizar Total da Mesa]
    B --> C{Aplicar Taxa de Serviço?}
    C -- Sim --> D[Adicionar 10%]
    C -- Não --> E[Remover Taxa]
    D --> F{Cliente tem Cupom?}
    E --> F
    F -- Sim --> G[Validar e Aplicar Desconto]
    F -- Não --> H[Selecionar Forma de Pagamento]
    G --> H
    H --> Pay
    Pay --> I[Registrar Recebimento]
    I --> J[Emitir Comprovante]
end

```

```

subgraph Sistema_Fin [Sistema Financeiro]
    G --> Val[Validar Validade Cupom]
    I --> Close[Alterar Status Mesa: Livre]
    Close --> K[Atualizar Fluxo de Caixa]
end

```

Pontos Chave Deste Diagrama:

Flexibilidade: O fluxo trata explicitamente a decisão da Taxa de Serviço e Cupons, requisitos explícitos do Módulo II.

Finalização: O sistema libera a mesa automaticamente após o pagamento.

Diagrama 3: Fluxo de Produção e Entrega (Cozinha)

Foco: Módulo I e Monitoramento. Foca no [RF04] Acompanhar Status do Pedido.

Snippet de código

```
graph TD
    subgraph Cozinha [Equipe da Cozinha]
        A([Pedido Recebido no Monitor]) --> B[Alterar Status: 'Em Preparação']
        B --> C[Executar Preparo]
        C --> D[Finalizar Prato]
        D --> E[Alterar Status: 'Pronto para Servir']
    end

    subgraph Sistema_Notificacao [Sistema / WebSocket]
        B --> Log1[Atualizar Log de Tempo]
        E --> Notify[Notificar Garçon Responsável]
    end

    subgraph Garcom [Garçon]
        Notify --> F[Receber Alerta]
        F --> G[Retirar Pedido na Passarela]
        G --> H([Entregar ao Cliente / Concluir])
    end
```

Pontos Chave Deste Diagrama:

Agilidade: Demonstra como o sistema elimina a necessidade do garçon ficar indo até a cozinha perguntar se o prato está pronto, cumprindo a missão de "Otimizar eficiência operacional".

Focando no Módulo de Pedidos e aplicando os princípios da Clean Architecture (que separa as regras de negócio de frameworks e UI), elaborei este Diagrama de Classes.

O objetivo aqui não é mostrar tabelas de banco de dados, mas sim as Entidades (o coração da regra de negócio) e os Casos de Uso (as ações que o sistema executa).

Diagrama de Classes: Domínio de Pedidos (Clean Architecture)

Snippet de código

```
classDiagram
    %% Camada de Entidades (Domain Layer)
    namespace Entities {
```

```

class Order {
    +String id
    +String tableId
    +OrderStatus status
    +List~OrderItem~ items
    +DateTime createdAt
    +addItem(item: OrderItem)
    +calculateTotal() Float
    +closeOrder()
}

class OrderItem {
    +String productId
    +String productName
    +Int quantity
    +Float unitPrice
    +subTotal() Float
}

class Product {
    +String id
    +String name
    +Float price
    +Int stockQuantity
    +hasStock(qtd: Int) Boolean
}

class Table {
    +String id
    +Int number
    +Boolean isOpen
    +open()
    +occupy()
}
}

%% Camada de Aplicação (Use Cases Layer)
namespace UseCases {
    class CreateOrderUseCase {
        -OrderRepository orderRepo
        -TableRepository tableRepo
        +execute(tableId: String) Order
    }

    class AddItemToOrderUseCase {
        -OrderRepository orderRepo
        -ProductRepository productRepo
        +execute(orderId: String, productId: String, qtd: Int)
    }
}

```

```
}  
}
```

%% Camada de Interfaces (Adapters/Gateways)

```
namespace Interfaces {  
  class IOrderRepository {  
    <<interface>>  
    +save(order: Order)  
    +findById(id: String) Order  
  }  
  
  class IProductRepository {  
    <<interface>>  
    +findById(id: String) Product  
    +updateStock(id: String, qtd: Int)  
  }  
}
```

%% Relacionamentos

Order *-- OrderItem : contém
Order --> Table : associado a
OrderItem --> Product : refere-se a

%% Use Cases usam as Entidades e Interfaces

CreateOrderUseCase ..> Order : cria
CreateOrderUseCase ..> Table : verifica
CreateOrderUseCase --> IOrderRepository : usa

AddItemToOrderUseCase ..> Order : atualiza
AddItemToOrderUseCase ..> Product : verifica estoque
AddItemToOrderUseCase --> IProductRepository : usa
AddItemToOrderUseCase --> IOrderRepository : usa

Explicação da Estrutura

Entities (Entidades): Estão no centro. Order, Product e Table contêm as regras críticas (ex: o método hasStock em Product ou calculateTotal em Order). Elas não sabem que o React ou Next.js existem.

Use Cases (Casos de Uso): São os "verbos" do sistema.

AddItemToOrderUseCase: Orquestra a operação. Ele busca o pedido, verifica se o produto tem estoque (usando a entidade Product), adiciona o item e salva.

Interfaces: Os Casos de Uso dependem de interfaces (IOrderRepository), não de implementações concretas (como um banco SQL ou Firebase). Isso facilita muito os testes.

Como você está usando Next.js, essa camada de Use Cases geralmente seria chamada pelas suas API Routes (ou Server Actions), que atuariam como os "Controllers".

Para manter a separação que desenhamos no diagrama, sugiro isolar o "Core" (Regras de Negócio) da infraestrutura do Next.js.

Aqui está a estrutura de pastas sugerida para o SaborRush:

Estrutura de Diretórios Sugerida

Plaintext

```
src/
├── app/                                # (Presentation Layer - Next.js)
│   ├── (public)/                       # Rotas públicas
│   ├── (admin)/                        # Rotas protegidas (Gestão)
│   │   ├── pedidos/
│   │   │   ├── page.tsx               # Tela de Pedidos (UI)
│   │   │   └── actions.ts             # Server Actions (atuam como Controllers)
│   │   └── cozinha/
│   ├── api/                            # API Routes (se precisar de endpoints REST externos)
│   └── layout.tsx
├── core/                               # (Domain & Application Layers - O "Coração")
│   ├── entities/                      # Entidades puras (sem React, sem DB)
│   │   ├── Order.ts                  # Classe Order
│   │   ├── Product.ts                # Classe Product
│   │   └── Table.ts                  # Classe Table
│   ├── use-cases/                    # Regras de aplicação
│   │   ├── orders/
│   │   │   ├── create-order.uc.ts    # CreateOrderUseCase
│   │   │   └── add-item.uc.ts        # AddItemToOrderUseCase
│   │   └── products/
│   ├── repositories/                 # Interfaces (Contratos)
│   │   ├── i-order.repository.ts
│   │   └── i-product.repository.ts
├── infra/                             # (Infrastructure Layer)
│   ├── database/                     # Implementação concreta do banco
│   │   ├── prisma/                  # Ex: Schema do Prisma
│   │   └── repositories/            # Implementação das interfaces
│   │       ├── prisma-order.repository.ts
│   │       └── memory-order.repository.ts (para testes)
│   └── adapters/                     # Adaptadores externos (ex: Pagamento, Email)
├── components/                       # (Presentation - UI Compartilhada)
│   ├── ui/                           # Botões, Inputs (ex: ShadcnUI)
│   ├── orders/                       # Componentes específicos de Pedido
│   └── order-card.tsx
```

Como o Diagrama se traduz nas Pastas?

Aqui está a relação direta entre o Diagrama de Classes que desenhamos e onde cada arquivo vive:

Entities (src/core/entities):

No diagrama: Order, Product.

No código: Classes TypeScript puras. Elas validam regras como "o preço não pode ser negativo" ou "calcular total". Regra de Ouro: Aqui nunca entra código de React (hooks) ou de Banco de Dados (Prisma/SQL).

Use Cases (src/core/use-cases):

No diagrama: CreateOrderUseCase.

No código: Funções ou classes que recebem os repositórios no construtor (Injeção de Dependência). É aqui que a mágica acontece. Exemplo: "Buscar produto > Verificar estoque > Salvar pedido".

Interfaces (src/core/repositories):

No diagrama: IOrderRepository.

No código: Interfaces TypeScript (interface IOrderRepository { ... }). Elas definem o que precisa ser salvo, mas não como.

Controllers (src/app/**/actions.ts):

No Next.js moderno (App Router), os Server Actions funcionam perfeitamente como Controllers.

O componente React (UI) chama uma Server Action. A Server Action instancia o Use Case, passa a implementação do repositório (ex: Prisma) e executa a ação.

Exemplo Prático de Integração (Server Action)

Imagine o arquivo src/app/(admin)/pedidos/actions.ts:

TypeScript

```
'use server'
```

```
// 1. Importa a implementação concreta (Infra)
import { PrismaOrderRepository } from
"@/infra/database/repositories/prisma-order.repository";
// 2. Importa o caso de uso (Core)
import { CreateOrderUseCase } from "@/core/use-cases/orders/create-order.uc";
```

```
export async function createAction(tableId: string) {  
  // Configuração da Injeção de Dependência  
  const orderRepo = new PrismaOrderRepository();  
  const createOrder = new CreateOrderUseCase(orderRepo);  
  
  // Execução  
  try {  
    const newOrder = await createOrder.execute(tableId);  
    return { success: true, data: newOrder };  
  } catch (error) {  
    return { success: false, error: error.message };  
  }  
}
```

Essa estrutura garante que se amanhã você quiser trocar o Next.js por uma API NestJS ou Express, ou trocar o banco de dados, você só mexe na pasta app ou infra. A pasta core (a inteligência do seu negócio) permanece intacta.